Union Find

AlgoExpert · Medium · Famous Algorithms

The problem

- Disjoint-set with createSet, find (null if unknown) and union, using path compression and union by rank.

Intuition

- Each set is a tree with a representative root; path compression and union by rank keep trees flat.

Approach

- 1. createSet makes a single-node tree
- 2. find follows parents with path compression
- 3. union attaches the shorter tree under the taller

Java solution

```java
public class UnionFind {
    private final Map<Integer, Integer> parent = new HashMap<>(), rank = new HashMap<>();

    public void createSet(int v) { parent.put(v, v); rank.put(v, 0); }

    public Integer find(int v) {
        if (!parent.containsKey(v)) return null;
        int p = parent.get(v);
        if (p != v) { p = find(p); parent.put(v, p); }
        return p;
    }

    public void union(int a, int b) {
        Integer ra = find(a), rb = find(b);
        if (ra == null || rb == null || ra.equals(rb)) return;
        int ka = rank.get(ra), kb = rank.get(rb);
        if (ka < kb) parent.put(ra, rb);
        else if (ka > kb) parent.put(rb, ra);
        else { parent.put(rb, ra); rank.put(ra, ka + 1); }
    }
}
```

Complexity

- Time: ~O(alpha(n)) amortised per op
- Space: O(n)

Recap

- Each set is a tree with a representative root; path compression and union by rank keep trees flat.
- Complexity: ~O(alpha(n)) amortised per op